

5. Списки и рендеринг массивов: `.map()` и `key`

Цель занятия

На прошлых занятиях мы научились создавать компоненты, передавать props и хранить состояние. Теперь разберем частую задачу: как отобразить несколько похожих элементов, не копируя JSX вручную.

В текущем проекте есть повторяющиеся ссылки:

javascript

```
<li>
  <a href="https://vite.dev/" target="_blank">
    <img className="logo" src={viteLogo} alt="" />
    Explore Vite
  </a>
</li>
<li>
  <a href="https://react.dev/" target="_blank">
    <img className="button-icon" src={reactLogo} alt="" />
    Learn more
  </a>
</li>
```

Такие элементы удобно хранить в массиве данных и отображать через `.map()`.

После занятия должны уметь:

- объяснить, зачем нужны массивы при рендеринге списков;
- создавать массив объектов для интерфейса;
- использовать `.map()` внутри JSX;
- возвращать JSX из `.map()`;
- понимать, зачем React нужен `key`;
- выбирать стабильный `key`;
- не использовать `Math.random()` как `key`;
- отличать данные списка от разметки списка.

1. Что говорит документация React

В актуальной документации React тема называется `Rendering Lists`. Главная идея: похожие компоненты можно создавать из массива данных с помощью JavaScript-методов, чаще всего `.map()` и иногда `.filter()`.

Коротко:

text

```
Массив хранит данные.  
.map() превращает данные в JSX.  
React отображает получившийся список.
```

Пример:

javascript

```
const names = ['Анна', 'Иван', 'Мария']  
  
function Students() {  
  return (  
    <ul>  
      {names.map((name) => (  
        <li key={name}>{name}</li>  
      ))}  
    </ul>  
  )  
}
```

Здесь массив `names` превращается в список `<li>`.

2. Почему не стоит копировать одинаковый JSX

Допустим, нужно вывести три ссылки:

javascript

```
<ul>  
  <li><a href="https://vite.dev/">Vite</a></li>  
  <li><a href="https://react.dev/">React</a></li>  
  <li><a href="https://developer.mozilla.org/">MDN</a></li>  
</ul>
```

Это работает, но у такого подхода есть минусы:

- сложно добавлять новые элементы;
- легко ошибиться при копировании;
- одинаковая структура повторяется несколько раз;
- данные смешаны с разметкой.

Лучше отделить данные от отображения.

Данные:

javascript

```
const links = [  
  { id: 'vite', label: 'Vite', href: 'https://vite.dev/' },  
  { id: 'react', label: 'React', href: 'https://react.dev/' },  
  { id: 'mdn', label: 'MDN', href: 'https://developer.mozilla.org/' },  
]
```

Отображение:

```
javascript
<ul>
  {links.map((link) => (
    <li key={link.id}>
      <a href={link.href}>{link.label}</a>
    </li>
  ))}
</ul>
```

Теперь, чтобы добавить ссылку, нужно добавить новый объект в массив.

3. Что такое массив

Массив - это список значений.

```
javascript
const numbers = [1, 2, 3]
const names = ['Анна', 'Иван', 'Мария']
```

В интерфейсах часто используют массив объектов:

```
javascript
const students = [
  { id: 1, name: 'Анна', group: '29-КД9-4ИНС' },
  { id: 2, name: 'Иван', group: '29-КД9-4ИНС' },
  { id: 3, name: 'Мария', group: '22-Д9-4ИНС' },
]
```

Каждый объект описывает один элемент будущего интерфейса.

4. Что такое `.map()`

`.map()` - метод массива, который проходит по каждому элементу и возвращает новый массив.

Простой пример:

```
javascript
const numbers = [1, 2, 3]
const doubled = numbers.map((number) => number * 2)
```

Результат:

```
javascript
[2, 4, 6]
```

В React мы часто превращаем массив данных в массив JSX-элементов:

```
javascript
const names = ['Анна', 'Иван', 'Мария']

const listItems = names.map((name) => (
  <li key={name}>{name}</li>
))
```

`listItems` будет массивом JSX-элементов.

5. `.map()` внутри JSX

В JSX можно вставлять JavaScript-выражения через фигурные скобки.

Поэтому можно написать так:

```
javascript
<ul>
  {names.map((name) => (
    <li key={name}>{name}</li>
  ))}
</ul>
```

Разберем:

```
javascript
names.map((name) => (
  <li key={name}>{name}</li>
))
```

Для каждого `name` создается один `<li>`.

Если `names` содержит три имени, React получит три элемента списка.

6. Почему нужен `key`

Когда React отображает список, ему нужно понимать, какой элемент списка какой.

Правильно:

```
javascript
{links.map((link) => (
  <li key={link.id}>
    <a href={link.href}>{link.label}</a>
  </li>
))}
```

`key` помогает React сопоставлять элементы между обновлениями.

Если элемент добавили, удалили или переставили, React по `key` понимает, что именно изменилось.

Главная идея:

```
text
key - это стабильный идентификатор элемента среди соседей.
```

7. Каким должен быть `key`

Хороший `key`:

- уникален среди элементов одного списка;
- не меняется при каждом рендере;
- берется из данных.

Хорошо:

```
javascript
{links.map((link) => (
  <li key={link.id}>{link.label}</li>
))}
```

Плохо:

```
javascript
{links.map((link) => (
  <li key={Math.random()}>{link.label}</li>
))}
```

Почему плохо? `Math.random()` каждый раз создает новое значение. React будет думать, что все элементы списка каждый раз новые.

Не лучший вариант:

```
javascript
{links.map((link, index) => (
  <li key={index}>{link.label}</li>
))}
```

Индекс иногда допустим для простых неизменяемых списков, но в проектах лучше сразу привыкать использовать стабильный `id`.

8. `key` не приходит как prop

Важно: `key` нужен React, но компонент не получает его как обычный prop.

Пример:

```
javascript
<LinkItem key={link.id} label={link.label} />
```

Внутри `LinkItem` нельзя прочитать `key`:

```
javascript
function LinkItem({ key, label }) {
  return <li>{label}</li>
}
```

Так делать не нужно.

Если компоненту нужен идентификатор, передайте его отдельно:

```
javascript
<LinkItem key={link.id} id={link.id} label={link.label} />
```

И получите:

```
javascript
function LinkItem({ id, label }) {
  return <li>{id}: {label}</li>
}
```

9. Массив объектов для ссылок

Для нашего проекта удобно создать массив:

```
javascript
const documentationLinks = [
  {
    id: 'vite',
    label: 'Explore Vite',
    href: 'https://vite.dev/',
    icon: viteLogo,
    iconClassName: 'logo',
  },
  {
    id: 'react',
    label: 'Learn more',
    href: 'https://react.dev/',
    icon: reactLogo,
    iconClassName: 'button-icon',
  },
]
```

Каждый объект описывает одну ссылку.

Поля объекта:

Поле	Что хранит
id	уникальный идентификатор для key
label	текст ссылки
href	адрес ссылки
icon	изображение
iconClassName	CSS-класс для изображения

10. Компонент элемента списка

Можно создать компонент для одной ссылки:

```
javascript
function ResourceLink({ href, label, icon, iconClassName }) {
  return (
    <li>
      <a href={href} target="_blank">
        <img className={iconClassName} src={icon} alt="" />
        {label}
      </a>
    </li>
  )
}
```

Теперь список можно собрать через `.map()`:

```
javascript
<ul>
  {documentationLinks.map((link) => (
    <ResourceLink
      key={link.id}
      href={link.href}
      label={link.label}
      icon={link.icon}
      iconClassName={link.iconClassName}
    />
  ))}
</ul>
```

11. Передача объекта через spread

Если имена полей объекта совпадают с props компонента, можно использовать spread-синтаксис:

```
javascript
<ResourceLink key={link.id} {...link} />
```

Это короткая запись вместо:

```
javascript
<ResourceLink
  key={link.id}
  href={link.href}
  label={link.label}
  icon={link.icon}
  iconClassName={link.iconClassName}
/>
```

spread можно использовать. Он удобен, но вы должны понимать, какие props реально передаются.

12. SVG-иконки из `public/icons.svg`

В проекте часть иконок берется не из `src/assets`, а из SVG-спрайта:

```
javascript
<svg className="button-icon" role="presentation" aria-hidden="true">
  <use href="/icons.svg#github-icon"></use>
</svg>
```

Для таких ссылок можно хранить `iconHref`:

```
javascript
const socialLinks = [
  {
    id: 'github',
    label: 'GitHub',
    href: 'https://github.com/vitejs/vite',
    iconHref: '/icons.svg#github-icon',
  },
]
```

И отображать:

```
javascript
<svg className="button-icon" role="presentation" aria-hidden="true">
  <use href={iconHref}></use>
</svg>
```

13. Один компонент для разных типов иконок

В нашем проекте есть два типа иконок:

- обычные изображения через `<img>`;
- SVG-иконки через `<svg><use /></svg>`.

Компонент может поддерживать оба варианта:

```
javascript
function ResourceLink({ href, label, icon, iconHref, iconClassName = 'button-icon' }) {
  return (
    <li>
      <a href={href} target="_blank">
        {icon ? (
          <img className={iconClassName} src={icon} alt="" />
        ) : (
          <svg className={iconClassName} role="presentation" aria-hidden="true">
            <use href={iconHref}></use>
          </svg>
        )}
        {label}
      </a>
    </li>
  )
}
```

Здесь используется условный рендеринг:

```
javascript
{icon ? <img /> : <svg />}
```

Если есть `icon`, показываем картинку. Если нет, показываем SVG-иконку.

14. Практический пример для проекта

Было:

```
javascript
<ul>
  <li>
    <a href="https://vite.dev/" target="_blank">
      <img className="logo" src={viteLogo} alt="" />
      Explore Vite
    </a>
  </li>
  <li>
    <a href="https://react.dev/" target="_blank">
      <img className="button-icon" src={reactLogo} alt="" />
      Learn more
    </a>
  </li>
</ul>
```

Стало:

```
javascript
const documentationLinks = [
  {
    id: 'vite',
    label: 'Explore Vite',
    href: 'https://vite.dev/',
    icon: viteLogo,
    iconClassName: 'logo',
  },
  {
    id: 'react',
    label: 'Learn more',
    href: 'https://react.dev/',
    icon: reactLogo,
  },
]
```

И:

```
javascript
<ul>
  {documentationLinks.map((link) => (
    <ResourceLink key={link.id} {...link} />
  ))}
</ul>
```

JSX стал короче, а данные стали видны отдельно.

15. `.filter()` - следующий шаг

Кроме `.map()`, React-документация рассказывает о `.filter()`.

`.filter()` оставляет только подходящие элементы.

```
javascript
const activeLinks = links.filter((link) => link.isActive)
```

Потом можно отрендерить:

```
javascript
{activeLinks.map((link) => (
  <ResourceLink key={link.id} {...link} />
))}
```

16. Частые ошибки новичков

Ошибка 1. Забыть `return` в `.map()`

Неправильно:

```
javascript
links.map((link) => {
  <li key={link.id}>{link.label}</li>
})
```

Фигурные скобки {} у стрелочной функции требуют явный return.

Правильно:

```
javascript
links.map((link) => {
  return <li key={link.id}>{link.label}</li>
})
```

Или короче:

```
javascript
links.map((link) => (
  <li key={link.id}>{link.label}</li>
))
```

Ошибка 2. Забыть `key`

Неправильно:

```
javascript
{links.map((link) => (
  <li>{link.label}</li>
))}
```

Правильно:

```
javascript
{links.map((link) => (
  <li key={link.id}>{link.label}</li>
))}
```

Ошибка 3. Использовать `Math.random()` как `key`

Неправильно:

```
javascript
<li key={Math.random()}>{link.label}</li>
```

Правильно:

```
javascript
<li key={link.id}>{link.label}</li>
```

Ошибка 4. Думать, что `key` доступен в props

Неправильно:

```
javascript
function ResourceLink({ key, label }) {
  return <li>{key}: {label}</li>
}
```

Правильно:

```
javascript
function ResourceLink({ id, label }) {
  return <li>{id}: {label}</li>
}
```

И передать отдельно:

```
javascript
<ResourceLink key={link.id} id={link.id} label={link.label} />
```

17. Практика

Задание 1

Создайте массив `documentationLinks`:

```
javascript
const documentationLinks = [
  {
    id: 'vite',
    label: 'Explore Vite',
    href: 'https://vite.dev/',
    icon: viteLogo,
    iconClassName: 'logo',
  },
  {
    id: 'react',
    label: 'Learn more',
    href: 'https://react.dev/',
    icon: reactLogo,
  },
]
```

Задание 2

Создайте компонент `ResourceLink`:

```
javascript
function ResourceLink({ href, label, icon, iconHref, iconClassName = 'button-icon' }) {
  return (
    <li>
      <a href={href} target="_blank">
        {icon ? (
          <img className={iconClassName} src={icon} alt="" />
        ) : (
          <svg className={iconClassName} role="presentation" aria-hidden="true">
            <use href={iconHref}></use>
          </svg>
        )}
        {label}
      </a>
    </li>
  )
}
```

Задание 3

Замените ручной список документации на `.map()`:

```
javascript
<ul>
  {documentationLinks.map((link) => (
    <ResourceLink key={link.id} {...link} />
  ))}
</ul>
```

Задание 4

Создайте массив `socialLinks` и перенесите туда ссылки GitHub, Discord, X.com и Bluesky.

Задание 5

Ответьте письменно:

- почему каждому элементу нужен `key`;
- почему `key={Math.random()}` - плохая идея;
- чем массив данных удобнее ручного копирования JSX.

18. Контрольные вопросы

1. Что такое массив?
2. Что такое объект в массиве данных?
3. Для чего нужен `.map()`?
4. Почему `.map()` удобно использовать в React?
5. Что возвращает `.map()` в JSX?
6. Зачем React нужен `key`?
7. Каким должен быть хороший `key`?
8. Почему не стоит использовать `Math.random()` как `key`?
9. Почему индекс массива не всегда хороший `key`?
10. Получает ли компонент `key` как обычный `prop`?
11. Как передать `id` отдельно, если он нужен компоненту?
12. Что такое `spread`-синтаксис `{...link}`?
13. Чем данные списка отличаются от разметки списка?
14. Для чего может пригодиться `.filter()`?
15. Почему массив объектов удобнее набора скопированных `<li>`?

19. Краткий итог

Списки в React удобно создавать из массивов данных. Метод `.map()` превращает каждый объект массива в JSX-элемент. Чтобы React правильно отслеживал элементы списка, каждому элементу нужен стабильный `key`.

Главная схема:

```
text
Массив данных
  ↓
.map()
  ↓
JSX-элементы
  ↓
Список в браузере
```

Главные правила:

text

- Похожие элементы лучше хранить в массиве данных.
- `.map()` создает JSX для каждого элемента.
- JSX внутри `.map()` должен иметь `key`.
- `key` должен быть стабильным и уникальным среди соседей.
- Не используем `Math.random()` для `key`.

Следующая логичная тема - условный рендеринг. Мы разберем, как показывать разные элементы интерфейса в зависимости от состояния, данных или условий.